UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS

Facultad de Ingeniería – Ciencias de la Computación

Teoría de Compiladores (1ACC0218) – 2026-2

Trabajo Final – Hito 2: Segundo avance

HospSim: un lenguaje de dominio específico para simular la atención hospitalaria

Sección: 4889
Profesor: JOSE LUIS SONCCO ALVAREZ

Integrantes

Apellidos y nombres	Código
Alvaro Jerry Rivera Calderón	u20231h075
Ericks Santiago Alarcon Castellanos	U202322704
Rodrigo Aaron Colonio Soto	u202323598

Repositorio: https://github.com/Jere348/HospSim

Lima, noviembre de 2026

Índice

1. Problemática y motivación.....3
2. Objetivos.....3
3. Construcciones del lenguaje.....4
4. Analizador léxico.....6
5. Analizador sintáctico.....8
6. Analizador semántico.....9
7. Arquitectura del compilador.....11
8. Plan de validación.....12
9. Estado del avance.....14
Referencias.....14
Anexo A. Derivaciones más a la izquierda.....15

1. Problemática y motivación

En los servicios de salud, especialmente en emergencias y consulta externa, el tiempo que un paciente espera depende de decisiones de capacidad: cuántos médicos asignar por turno, cuántas camas habilitar o cada cuánto pueden llegar pacientes sin saturar el servicio. Estas decisiones suelen tomarse por experiencia o con hojas de cálculo que no reflejan cómo se forman las colas cuando varios pacientes compiten por el mismo recurso.

La simulación de eventos discretos es la técnica estándar para responder estas preguntas, pero las herramientas disponibles tienen una barrera de entrada alta: los paquetes comerciales requieren licencias y capacitación, y las bibliotecas de propósito general (por ejemplo, SimPy en Python) exigen saber programar. El resultado es que quienes conocen el proceso (jefes de servicio, administradores de salud) no pueden describirlo por sí mismos, y quienes saben programar no conocen el proceso. Construir un modelo se convierte en un proceso largo de idas y vueltas.

HospSim propone un lenguaje de dominio específico, escrito en español y con vocabulario hospitalario (recurso, proceso, etapa, pacientes), en el que un programa de pocas líneas describe los recursos, los flujos de atención, la llegada de pacientes y el horizonte de simulación, y obtiene directamente métricas como pacientes atendidos, espera promedio y porcentaje de ocupación de cada recurso. El compilador detecta antes de ejecutar los errores de modelado más comunes (recursos inexistentes, capacidades inválidas, tiempos no positivos), algo que una hoja de cálculo no hace.

```
recurso medicos = 1;
proceso atencion {
    etapa consulta usa medicos durante 15 minutos;
}
generar pacientes cada 5 minutos hacia atencion;
simular durante 1 hora;
mostrar pacientes_atendidos;      // 4
mostrar espera_promedio;          // 15.00 minutos
mostrar ocupacion(medicos);       // 100.00 %
```

Programa 1. Ejemplo mínimo en HospSim y los resultados que produce el compilador.

2. Objetivos

2.1 Objetivo general

Diseñar e implementar un compilador para HospSim que traduzca descripciones de procesos hospitalarios en una simulación ejecutable y reporte métricas de desempeño.

2.2 Objetivos específicos

- Definir el léxico del lenguaje e implementarlo como analizador léxico en ANTLR4.
- Diseñar una gramática libre de contexto, no ambigua, para cada construcción del lenguaje e implementarla como analizador sintáctico en ANTLR4.
- Implementar un analizador semántico con tabla de símbolos por ámbitos que verifique al menos seis errores semánticos (se verifican 21).
- Construir una representación intermedia independiente de ANTLR, emitir su versión textual y ejecutar el modelo mediante una simulación por eventos discretos.
- Validar el compilador con pruebas automáticas cuyas salidas esperadas se calculan a mano.

3. Construcciones del lenguaje

Un programa HospSim es una secuencia de sentencias. Los identificadores deben declararse antes de usarse; los comentarios empiezan con // y llegan hasta el fin de la línea. Las unidades de tiempo admitidas son minuto, minutos, hora y horas; internamente todo se normaliza a minutos. A continuación se describen las siete construcciones con cinco ejemplos cada una (todos los ejemplos fueron verificados con el analizador sintáctico).

3.1 Declaración de recursos

Forma: `recurso id = expresión ;`

Declara un recurso del hospital (médicos, camas, equipos) y su capacidad: cuántos pacientes puede atender a la vez.

N°	Ejemplo
1	`recurso medicos = 3;`
2	`recurso camas = 20;`
3	`recurso enfermeros = medicos * 2;`
4	`recurso boxes_triaje = (camas - 4) / 4;`
5	`recurso tecnicos_rx = 1;`

3.2 Declaración de procesos

Forma: `proceso id { etapas }`

Define un flujo de atención como una secuencia ordenada de etapas. Cada proceso abre un ámbito propio.

N°	Ejemplo
1	`proceso consulta_externa { etapa consulta usa medicos durante 20 minutos; }`
2	`proceso emergencia { etapa triaje usa enfermeros durante 5 minutos; etapa atencion usa medicos durante 30 minutos; }`
3	`proceso hospitalizacion { etapa admision usa admisionistas durante 15 minutos; etapa internamiento usa camas durante 48 horas; }`
4	`proceso imagenes { etapa radiografia usa tecnicos_rx durante 10 minutos; etapa informe usa radiologos durante 0.5 horas; }`
5	`proceso cirugia_menor { etapa preparacion usa enfermeros durante 20 minutos; etapa operacion usa cirujanos durante 1 hora; etapa recuperacion usa camas durante 2 horas; }`

3.3 Declaración de etapas

Forma: `etapa id usa id durante expresión unidad ;`

Dentro de un proceso, indica qué recurso ocupa el paciente y por cuánto tiempo.

N°	Ejemplo
1	`etapa triaje usa enfermeros durante 5 minutos;`
2	`etapa consulta usa medicos durante 15 minutos;`
3	`etapa internamiento usa camas durante 2 * 24 horas;`
4	`etapa laboratorio usa tecnicos durante 45 minutos;`

N°	Ejemplo
5	`etapa alta usa admision durante 1 hora;`

3.4 Generación de llegadas

Forma: `generar pacientes cada expresión unidad hacia id ;`

Indica cada cuánto tiempo llega un paciente nuevo y a qué proceso ingresa.

N°	Ejemplo
1	`generar pacientes cada 5 minutos hacia emergencia;`
2	`generar pacientes cada 12 minutos hacia consulta_externa;`
3	`generar pacientes cada 1 hora hacia hospitalizacion;`
4	`generar pacientes cada 0.5 horas hacia imagenes;`
5	`generar pacientes cada 60 / 4 minutos hacia emergencia;`

3.5 Simulación

Forma: `simular durante expresión unidad ;`

Fija el horizonte de tiempo de la simulación. Debe aparecer exactamente una vez en el programa.

N°	Ejemplo
1	`simular durante 8 horas;`
2	`simular durante 12 horas;`
3	`simular durante 90 minutos;`
4	`simular durante 1 hora;`
5	`simular durante 24 * 7 horas;`

3.6 Consulta de métricas

Forma: `mostrar id ;  |  mostrar id ( id ) ;`

Solicita un resultado de la simulación: pacientes atendidos, espera promedio u ocupación de un recurso.

N°	Ejemplo
1	`mostrar pacientes_atendidos;`
2	`mostrar espera_promedio;`
3	`mostrar ocupacion(medicos);`
4	`mostrar ocupacion(camas);`
5	`mostrar ocupacion(enfermeros);`

3.7 Expresiones aritméticas

Forma: `E → E + T | E − T | T ...`

Números enteros o decimales, identificadores de recursos (aportan su capacidad), + − * / y paréntesis, con la precedencia y asociatividad usuales.

N°	Ejemplo
1	`15`
2	`2 * 10`
3	`medicos + 4`
4	`(camas - 4) / 2`
5	`24 * 7 - 8`

3.8 Programa completo

```
// ejemplo completo
recurso medicos = 2 * 1;
recurso camas = 5;
proceso atencion {
  etapa triaje usa medicos durante 10 minutos;
  etapa cama usa camas durante 1 hora;
}
generar pacientes cada 5 minutos hacia atencion;
simular durante 8 horas;
mostrar pacientes_atendidos;
mostrar ocupacion(medicos);
mostrar ocupacion(camas);
mostrar espera_promedio;
```

Programa 2. Emergencia con triaje y hospitalización (tests/entrada_completa.txt).

```
Análisis semántico: OK
=== RESULTADOS ===
  pacientes_atendidos = 35
  ocupacion(medicos) = 99.48 %
  ocupacion(camas) = 95.83 %
  espera_promedio = 105.00 minutos
```

Salida del compilador para el Programa 2.

4. Analizador léxico

4.1 Léxico del lenguaje (token: lista de lexemas)

HospSim tiene un único tipo de dato numérico (entero o decimal), usado para capacidades y tiempos. Los identificadores toman su tipo de la declaración: recurso, proceso, etapa o métrica. Las métricas pacientes_atendidos, espera_promedio y ocupacion no son palabras reservadas: son identificadores predeclarados en el ámbito global de la tabla de símbolos.

Token	Lexemas	Descripción / patrón
`RECURSO, PROCESO, ETAPA`	`recurso, proceso, etapa`	Palabras reservadas de declaración
`USA, DURANTE, HACIA, CADA`	`usa, durante, hacia, cada`	Palabras reservadas de enlace
`GENERAR, PACIENTES`	`generar, pacientes`	Palabras reservadas de llegada
`SIMULAR, MOSTRAR`	`simular, mostrar`	Palabras reservadas de ejecución y consulta
`MINUTO, MINUTOS`	`minuto, minutos`	Unidad de tiempo (×1)
`HORA, HORAS`	`hora, horas`	Unidad de tiempo (×60)
`ASSIGN`	`=`	Asignación de capacidad
`SEMI`	`;`	Fin de sentencia

Token	Lexemas	Descripción / patrón
LBRACE, RBRACE	{, }	Delimitan el cuerpo de un proceso
LPAREN, RPAREN	(,)	Agrupación y argumento de métrica
PLUS, MINUS	+, -	Operadores aditivos
MULT, DIV	*, /	Operadores multiplicativos
ID	medicos, camas, consulta_externa, triaje_1, ocupacion	[a-zA-Z_][a-zA-Z0-9_]*
NUMBER	0, 15, 48, 2.5, 0.5	[0-9]+ ('.' [0-9]+)?
WS (se descarta)	espacio, \t, \r, \n	[\t\r\n]+
COMMENT (se descarta)	// texto hasta fin de línea	'//' ~[\r\n]*

4.2 Implementación en ANTLR4

Las reglas léxicas están en HospSim.g4. Cada palabra reservada se declara antes que ID; ante dos reglas que reconocen el mismo lexema de igual longitud, ANTLR elige la primera, por eso "proceso" es PROCESO y no ID. La regla de la coincidencia más larga hace que "procesos" sea un ID y que "minutos" no se lea como "minuto" seguido de "s". WS y COMMENT se descartan con -> skip.

4.3 Pruebas

El driver tiene un modo que imprime la secuencia de tokens (python3 main.py --tokens archivo). Para el Programa 1:

```
1:0    RECURSO    'recurso'
1:8    ID         'medicos'
1:16   ASSIGN     '='
1:18   NUMBER     '1'
1:19   SEMI       ';'
3:0    PROCESO    'proceso'
3:8    ID         'atencion'
3:17   LBRACE     '{'
4:4    ETAPA      'etapa'
4:10   ID         'consulta'
4:19   USA        'usa'
4:23   ID         'medicos'
4:31   DURANTE    'durante'
4:39   NUMBER     '15'
4:42   MINUTOS    'minutos'
4:49   SEMI       ';'
5:0    RBRACE     '}'
...
```

Con un carácter que no pertenece al lenguaje (recurso m = 1 @ 2;), el analizador léxico informa línea y columna:

```
1:0    RECURSO    'recurso'
1:8    ID         'm'
1:10   ASSIGN     '='
1:12   NUMBER     '1'
1:16   NUMBER     '2'
1:17   SEMI       ';'
  [Error Léxico, línea 1:14] token recognition error at: '@'
```

5. Analizador sintáctico

5.1 Gramática (notación del curso)

Se sigue la convención de notación de clase: los no-terminales se escriben en *cursiva* (E, T y F representan expresión, término y factor), los terminales en **negrita** junto con los operadores y signos de puntuación, ε es la cadena vacía y la cabeza de la primera producción, *programa*, es el símbolo inicial. **id** y **num** representan los tokens ID y NUMBER.

programa	→	*sentencias*
sentencias	→	*sentencia sentencias* | ε
sentencia	→	*decl_recurso* | *decl_proceso* | *sent_llegada* | *sent_simular* | *sent_mostrar*
decl_recurso	→	**recurso id** = *E* ;
decl_proceso	→	**proceso id** { *etapas* }
etapas	→	*decl_etapa etapas* | ε
decl_etapa	→	**etapa id usa id durante** *E unidad* ;
sent_llegada	→	**generar pacientes cada** *E unidad* **hacia id** ;
sent_simular	→	**simular durante** *E unidad* ;
sent_mostrar	→	**mostrar id** ; | **mostrar id** (**id**) ;
unidad	→	**minuto** | **minutos** | **hora** | **horas**
E	→	*E* + *T* | *E* − *T* | *T*
T	→	*T* * *F* | *T* / *F* | *F*
F	→	(*E*) | **num** | **id**

Gramática G de HospSim. Las listas X de ANTLR se escriben con recursión y ε (sentencias, etapas).*

5.2 Gramática por construcción

Cada construcción del lenguaje tiene su propia gramática, formada por su no-terminal y las producciones que alcanza: programa (programa, sentencias, sentencia), declaración de recurso (decl_recurso + expresiones), declaración de proceso (decl_proceso, etapas, decl_etapa, unidad + expresiones), llegada (sent_llegada), simulación (sent_simular), consulta (sent_mostrar) y expresiones (E, T, F). En el Anexo A se presentan cuatro derivaciones más a la izquierda para cada una (28 en total).

5.3 Gramática actualizada respecto al Hito 1

En el Hito 1 las expresiones usaban una sola regla ambigua, expr → expr (+|−) expr | expr (*|/) expr | …, con la suma escrita antes que la multiplicación. ANTLR resuelve esa ambigüedad dando mayor precedencia a la alternativa que aparece primero, por lo que 2 + 3 * 4 se agrupaba como (2 + 3) * 4 = 20. En el Hito 2 la regla se reemplazó por la gramática estratificada E, T, F vista en clase, que es no ambigua: fija la precedencia de * y / sobre + y −, y la asociatividad por la izquierda (20 − 5 − 3 = 12; 24 / 4 / 2 = 3). Ambos casos tienen una prueba automática (sim_precedencia y sim_asociatividad).

```
// Gramática estratificada: expr (E) → term (T) → factor (F).
// La precedencia (* / sobre + -) y la asociatividad izquierda quedan fijadas por
la gramática.
expr
    : expr (PLUS | MINUS) term     # AddExpr
    | term                         # TermExpr
    ;
```

```
term
    : term (MULT | DIV) factor     # MultExpr
    | factor                       # FactorExpr
    ;

factor
    : NUMBER                       # NumExpr
    | ID                           # IdExpr
    | LPAREN expr RPAREN           # ParenExpr
    ;
```

Reglas de expresiones en HospSim.g4. Las etiquetas (# AddExpr, …) generan un método visit por alternativa.

5.4 Ejemplo de derivación más a la izquierda

La entrada 2 + 3 * 4 tiene una única derivación más a la izquierda en G; en ella la multiplicación queda más profunda en el árbol que la suma, por lo que se evalúa primero:

Entrada: `2 + 3 * 4`

E	
$\Rightarrow_{lm}$ *E* + *T*	E → E + T
$\Rightarrow_{lm}$ *T* + *T*	E → T
$\Rightarrow_{lm}$ *F* + *T*	T → F
$\Rightarrow_{lm}$ **num** + *T*	F → num
$\Rightarrow_{lm}$ **num** + *T* * *F*	T → T * F
$\Rightarrow_{lm}$ **num** + *F* * *F*	T → F
$\Rightarrow_{lm}$ **num** + **num** * *F*	F → num
$\Rightarrow_{lm}$ **num** + **num** * **num**	F → num

5.5 Implementación y pruebas

El parser se genera con ANTLR 4.13.1 (make generate) en Python 3 con la opción -visitor. El driver reemplaza los listeners de error por defecto por un colector propio, de modo que los errores se reportan en español con línea y columna y la compilación se detiene antes del análisis semántico. Para tests/error_sintactico.txt (falta un ; y el nombre del proceso):

```
=== ERRORES LÉXICOS / SINTÁCTICOS ===
  [Error Sintáctico, línea 2:0] missing ';' at 'proceso'
  [Error Sintáctico, línea 2:8] missing ID at '{'
```

6. Analizador semántico

6.1 Tabla de símbolos

El análisis semántico es un visitor (SemanticVisitor) que recorre el árbol una vez, en orden. La tabla de símbolos guarda, para cada ámbito, los símbolos declarados con su nombre, tipo (recurso, proceso, etapa o métrica), ámbito, línea, valor (capacidad de los recursos) y si fue usado. Hay un ámbito global y uno por cada proceso; la búsqueda (lookup) recorre la pila de ámbitos del más interno al global. Las tres métricas se predeclaran en el ámbito global. Las expresiones se evalúan en tiempo de compilación: un recurso usado dentro de una expresión aporta su capacidad, lo que permite validar valores como recurso n = m − 2.

6.2 Errores semánticos verificados

El Hito 2 exige al menos seis errores semánticos; el compilador verifica 21. Los mensajes de la tabla son la salida real del compilador para un programa mínimo que contiene solo ese error.

N°	Verificación	Ejemplo	Mensaje del compilador
1	Identificador duplicado en el mismo ámbito	`recurso m = 2;`	[línea 2] El identificador 'm' ya ha sido declarado en el ámbito 'global'.
2	Recurso no declarado en una etapa	`etapa e usa enfermeros durante 5 minutos;`	[línea 2] Recurso 'enfermeros' utilizado en etapa 'e' no está declarado
3	Lo usado en una etapa no es un recurso	`etapa x usa p durante 5 minutos;`	[línea 3] 'p' es de tipo 'proceso'; una etapa solo puede usar un recurso
4	Proceso destino no declarado	`... hacia fantasma;`	[línea 4] El proceso de destino 'fantasma' no ha sido declarado
5	El destino de una llegada no es un proceso	`... hacia m;`	[línea 4] 'm' no es un proceso válido de destino
6	Identificador no declarado en una expresión	`recurso n = x + 1;`	[línea 2] El identificador 'x' no está declarado
7	Identificador que no es recurso en una expresión	`recurso n = p * 2;`	[línea 3] 'p' es de tipo 'proceso'; en una expresión solo se pueden usar recursos
8	División entre cero	`recurso n = 4 / (2 - 2);`	[línea 2] División entre cero
9	Capacidad menor o igual a 0	`recurso n = 2 - 5;`	[línea 2] La capacidad del recurso 'n' debe ser mayor a 0
10	Capacidad no entera	`recurso n = 3 / 2;`	[línea 2] La capacidad del recurso 'n' debe ser un número entero (se obtuvo 1.5)
11	Duración de etapa menor o igual a 0	`etapa e usa m durante 0 minutos;`	[línea 2] La duración de la etapa 'e' debe ser mayor a 0
12	Intervalo de llegada menor o igual a 0	`generar pacientes cada 0 minutos hacia p;`	[línea 3] El intervalo de llegada de pacientes debe ser mayor a 0
13	Duración de simulación menor o igual a 0	`simular durante 0 horas;`	[línea 4] La duración de la simulación debe ser mayor a 0
14	Proceso sin etapas	`proceso q { }`	[línea 3] El proceso 'q' no tiene etapas
15	Falta la sentencia simular	`(programa sin simular)`	[línea 3] Falta la sentencia 'simular durante ...'
16	Más de una sentencia simular	`simular ... (segunda vez)`	[línea 5] Solo se permite una sentencia 'simular'
17	mostrar sobre algo que no es métrica	`mostrar m;`	[línea 5] 'm' no es una métrica (use pacientes_atendidos, espera_promedio u ocupacion)
18	ocupacion sin recurso	`mostrar ocupacion;`	[línea 5] La métrica 'ocupacion' requiere un recurso: ocupacion(recurso)
19	Métrica que no recibe argumentos	`mostrar espera_promedio(m);`	[línea 5] La métrica 'espera_promedio' no recibe argumentos
20	ocupacion sobre algo que no es recurso	`mostrar ocupacion(p);`	[línea 5] No se puede medir ocupación sobre 'p' porque no es un recurso
21	ocupacion sobre un recurso inexistente	`mostrar ocupacion(rayos_x);`	[línea 5] El recurso 'rayos_x' no existe para la consulta de ocupación

Además, se emite una advertencia (no detiene la compilación) por cada recurso o proceso declarado que nunca se usa. Si hay errores, la compilación termina con código 1 y no se ejecuta el back end.

7. Arquitectura del compilador

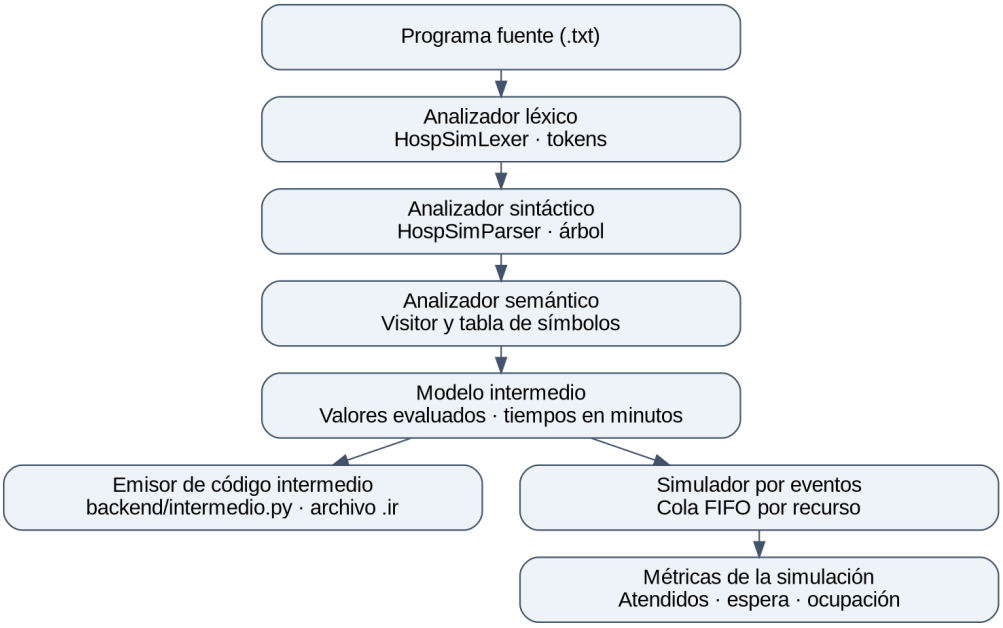


Figura 1. Fases del compilador HospSim.

Módulo	Responsabilidad
`HospSim.g4`	Gramática léxica y sintáctica (ANTLR4).
`gen/`	Lexer, parser y visitor generados por ANTLR 4.13.1.
`main.py`	Driver: encadena las fases, reporta errores con línea y columna y define el código de salida (0 correcto, 1 con errores).
`semantic/ symbol_table.py`	Tabla de símbolos con pila de ámbitos; advertencias de símbolos sin usar.
`semantic/ semantic_visitor.py`	Verificaciones semánticas y evaluación de expresiones constantes.
`backend/modelo.py`	Representación intermedia: recursos, procesos y etapas, llegadas, duración y consultas, con valores evaluados y tiempos en minutos.
`backend/simulador.py`	Back end del Hito 2: simulación determinista y reporte de métricas.
`tools/derivaciones.py`	Genera las derivaciones más a la izquierda del informe a partir del parser real.
`tests/, run_tests.sh`	Casos de prueba con salida esperada y ejecución automática.
backend/intermedio.py	Emite instrucciones RESOURCE, QUEUE, PROCESS, ARRIVALS, RUN_UNTIL, REPORT, ACQUIRE, DELAY, RELEASE y COMPLETE.

7.1 Decisiones de diseño

- Recorridos separados: el back end solo recibe árboles que pasaron el análisis semántico, por lo que no repite validaciones.
- Representación intermedia propia: el árbol validado se transforma en un Modelo con valores evaluados y tiempos en minutos. backend/intermedio.py emite las instrucciones textuales; backend/simulador.py ejecuta el mismo Modelo. El archivo .ir es una salida de compilación y no se vuelve a analizar para ejecutar. El alcance aprobado requiere código intermedio; no requiere LLVM.

- Simulación por eventos discretos: los pacientes llegan en t = 0, k, 2k, … mientras t sea menor que la duración. Cada recurso tiene una cola FIFO compartida por los procesos que lo usan. Solo se solicita un recurso cuando el paciente alcanza esa etapa. Se contabilizan las finalizaciones que ocurren hasta el horizonte, inclusive.
- Reglas de desempate: a igual tiempo, se procesan primero las finalizaciones y luego las llegadas. Dentro de cada clase se mantiene el orden de inserción de los eventos; las llegadas simultáneas iniciales siguen el orden de declaración. Los pacientes que ya esperan tienen prioridad. La espera promedio incluye solo a quienes completaron el proceso y la ocupación considera también el servicio parcial dentro del horizonte.

7.2 Código intermedio generado

El comando python main.py --ir ejemplos/atencion.txt genera la siguiente salida. Para guardarla: python main.py --ir --guardar-ir salida.ir ejemplos/atencion.txt. Las instrucciones de proceso se activan por ARRIVALS; HALT marca el fin del bloque principal.

```
; HospSim IR 1 - tiempos en minutos
RESOURCE medicos 1
QUEUE medicos FIFO
PROCESS atencion atencion_inicio
ARRIVALS atencion EVERY 5 FROM 0
RUN_UNTIL 60
REPORT pacientes_atendidos
REPORT espera_promedio
REPORT ocupacion medicos
HALT
atencion_inicio:
  ; etapa consulta
  ACQUIRE medicos
  DELAY 15
  RELEASE medicos
  COMPLETE
```

8. Plan de validación

8.1 Método

Se usan 20 casos de caja negra con salida esperada y 11 pruebas de regresión. python run_tests.py funciona en Windows y Linux: compara la salida y el código de retorno, y ejecuta tests/test_regresiones.py. Las regresiones comprueban recursos compartidos, colas FIFO, reentrada, límites del horizonte, capacidades, ámbitos y código intermedio. Las salidas esperadas se contrastan con cálculos independientes.

8.2 Casos de prueba

Fase	Casos	Qué se verifica
Léxica	`error_lexico`	Carácter inválido reportado con línea y columna.
Sintáctica	`error_sintactico`	Punto y coma e identificador faltantes.
Semántica	`entrada_error, error_capacidad, error_capacidad_entera, error_duplicado, error_estructura, error_expr, error_metricas, error_simular_doble, error_tiempos, error_tipo_expresion, error_tipos, error_vacio`	Errores semánticos y advertencias de símbolos sin usar.

Fase	Casos	Qué se verifica
Gramática	`sim_precedencia, sim_asociatividad`	Precedencia (2 + 3 * 4 = 14) y asociatividad izquierda (20 − 5 − 3 = 12).
Back end	`entrada_correcta, sim_dos_servidores, sim_dos_etapas, sim_precedencia, sim_asociatividad, entrada_completa`	Métricas comparadas con cálculos a mano.

8.3 Resultados esperados calculados a mano

Caso	Cálculo	Resultado
`entrada_correcta`	1 médico, consulta 15 min, llegada cada 5 min, 60 min. Atenciones 0–15, 15–30, 30–45, 45–60; la quinta termina en 75. Esperas 0, 10, 20, 30.	4 atendidos; espera 15.00; ocupación 100.00 %
`sim_dos_servidores`	Igual con 2 médicos. Servidor A: 0–60 continuo; B: 5–65 (55 min dentro del horizonte). Esperas 0, 0, 5, 5, 10, 10, 15.	7; 45/7 = 6.43; 115/120 = 95.83 %
`sim_dos_etapas`	Triaje 5 min (1 enfermero) y consulta 12 min (1 médico), llegada cada 10 min. Consultas 5–17, 17–29, 29–41, 41–53; esperas 0, 2, 4, 6.	4; 3.00; 50.00 % y 55/60 = 91.67 %
`sim_precedencia`	14 camas, 60 min, llegada cada 30 min, 120 min. Fines 60, 90, 120, 150. Con la gramática del Hito 1 serían 20 camas y 8.75 %.	3; 210/1680 = 12.50 %
`sim_asociatividad`	12 médicos y 3 camas; llegada cada hora, 180 min. Médicos 180 min de uso, camas 60 min.	2; 180/2160 = 8.33 %; 60/540 = 11.11 %
`entrada_completa`	2 médicos (10 min) y 5 camas (60 min), llegada cada 5 min, 480 min. El paciente i = 5q + r entra a cama en 5r + 10 + 60q y espera 35q; terminan a tiempo q = 0…6.	35; 105.00; 955/960 = 99.48 %; 2300/2400 = 95.83 %

8.4 Resultados de la ejecución

Los 20 casos originales coinciden con sus salidas esperadas. También pasan las 11 pruebas de regresión de tests/test_regresiones.py. Total: 31 pruebas satisfactorias.

```
OK     entrada_completa
OK     entrada_correcta
OK     entrada_error
OK     error_capacidad
OK     error_capacidad_entera
OK     error_duplicado
OK     error_estructura
OK     error_expr
OK     error_lexico
OK     error_metricas
OK     error_simular_doble
OK     error_sintactico
OK     error_tiempos
OK     error_tipo_expresion
OK     error_tipos
OK     error_vacio
OK     sim_asociatividad
OK     sim_dos_etapas
OK     sim_dos_servidores
OK     sim_precedencia
0 prueba(s) fallida(s)
```

Criterio de aceptación: los 20 casos de salida y las 11 regresiones deben pasar. El escenario de recurso compartido verifica 1 paciente atendido y 10 % de ocupación del recurso b en 10 minutos. Las capacidades cercanas a cero se rechazan, el proceso llamado global conserva un ámbito independiente y la salida .ir coincide con el ejemplo esperado.

9. Estado del avance

Componente	Estado
Analizador léxico (ANTLR4)	Completo
Analizador sintáctico (ANTLR4, gramática estratificada)	Completo
Analizador semántico (21 verificaciones, tabla de símbolos por ámbitos)	Completo
Representación intermedia (Modelo)	Completa
Back end de simulación (intérprete del Modelo)	Completo
Pruebas automáticas con salidas verificadas a mano	20 casos originales y 11 regresiones, todos en OK
Emisión de código intermedio textual (.ir)	Implementada: --ir y --guardar-ir

El avance incluye análisis léxico, sintáctico y semántico, emisión textual de código intermedio y ejecución mediante un simulador por eventos. Para la entrega final se ampliarán los escenarios de validación y se revisarán los límites de rendimiento. La simulación es determinista: no incorpora prioridades clínicas, rutas condicionales ni tiempos aleatorios.

Referencias

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). Compilers: Principles, techniques, and tools (2.ª ed.). Pearson.

Banks, J., Carson, J. S., Nelson, B. L., & Nicol, D. M. (2010). Discrete-event system simulation (5.ª ed.). Pearson.

Parr, T. (2013). The definitive ANTLR 4 reference. The Pragmatic Bookshelf.

Anexo A. Derivaciones más a la izquierda

Generadas con tools/derivaciones.py a partir del árbol que construye el parser de ANTLR, convertido a la gramática G de la sección 5.1. El script verifica que toda producción usada pertenezca a G. A la derecha de cada paso se indica la producción aplicada.

A.1 Programa

Entrada: `simular durante 8 horas;`

programa

⇒lm *sentencias*	programa → sentencias
⇒lm *sentencia sentencias*	sentencias → sentencia sentencias
⇒lm *sent_simular sentencias*	sentencia → sent_simular
⇒lm **simular durante** *E unidad* ; *sentencias*	sent_simular → simular durante E unidad ;
⇒lm **simular durante** *T unidad* ; *sentencias*	E → T
⇒lm **simular durante** *F unidad* ; *sentencias*	T → F
⇒lm **simular durante num** *unidad* ; *sentencias*	F → num
⇒lm **simular durante num horas** ; *sentencias*	unidad → horas
⇒lm **simular durante num horas** ;	sentencias → ε

Entrada: `recurso medicos = 2;  simular durante 1 hora;`

programa

⇒lm *sentencias*	programa → sentencias
⇒lm *sentencia sentencias*	sentencias → sentencia sentencias
⇒lm *decl_recurso sentencias*	sentencia → decl_recurso
⇒lm **recurso id** = *E* ; *sentencias*	decl_recurso → recurso id = E ;
⇒lm **recurso id** = *T* ; *sentencias*	E → T
⇒lm **recurso id** = *F* ; *sentencias*	T → F
⇒lm **recurso id** = **num** ; *sentencias*	F → num
⇒lm **recurso id** = **num** ; *sentencia sentencias*	sentencias → sentencia sentencias
⇒lm **recurso id** = **num** ; *sent_simular sentencias*	sentencia → sent_simular
⇒lm **recurso id** = **num** ; **simular durante** *E unidad* ; *sentencias*	sent_simular → simular durante E unidad ;
⇒lm **recurso id** = **num** ; **simular durante** *T unidad* ; *sentencias*	E → T
⇒lm **recurso id** = **num** ; **simular durante** *F unidad* ; *sentencias*	T → F
⇒lm **recurso id** = **num** ; **simular durante num** *unidad* ; *sentencias*	F → num
⇒lm **recurso id** = **num** ; **simular durante num hora** ; *sentencias*	unidad → hora
⇒lm **recurso id** = **num** ; **simular durante num hora** ;	sentencias → ε

Entrada: `recurso camas = 10;  mostrar ocupacion(camas);`

programa

⇒ₗₘ *sentencias*	programa → sentencias
⇒ₗₘ *sentencia sentencias*	sentencias → sentencia sentencias
⇒ₗₘ *decl_recurso sentencias*	sentencia → decl_recurso
⇒ₗₘ **recurso id** = *E* ; *sentencias*	decl_recurso → recurso id = E ;
⇒ₗₘ **recurso id** = *T* ; *sentencias*	E → T
⇒ₗₘ **recurso id** = *F* ; *sentencias*	T → F
⇒ₗₘ **recurso id** = **num** ; *sentencias*	F → num
⇒ₗₘ **recurso id** = **num** ; *sentencia sentencias*	sentencias → sentencia sentencias
⇒ₗₘ **recurso id** = **num** ; *sent_mostrar sentencias*	sentencia → sent_mostrar
⇒ₗₘ **recurso id** = **num** ; **mostrar id** (**id**) ; *sentencias*	sent_mostrar → mostrar id (id) ;
⇒ₗₘ **recurso id** = **num** ; **mostrar id** (**id**) ;	sentencias → ε

Entrada: `proceso p { }  generar pacientes cada 5 minutos hacia p;`

programa

⇒ₗₘ *sentencias*	programa → sentencias
⇒ₗₘ *sentencia sentencias*	sentencias → sentencia sentencias
⇒ₗₘ *decl_proceso sentencias*	sentencia → decl_proceso
⇒ₗₘ **proceso id** { *etapas* } *sentencias*	decl_proceso → proceso id { etapas }
⇒ₗₘ **proceso id** { } *sentencias*	etapas → ε
⇒ₗₘ **proceso id** { } *sentencia sentencias*	sentencias → sentencia sentencias
⇒ₗₘ **proceso id** { } *sent_llegada sentencias*	sentencia → sent_llegada
⇒ₗₘ **proceso id** { } **generar pacientes cada** *E unidad* **hacia id** ; *sentencias*	sent_llegada → generar pacientes cada E unidad hacia id ;
⇒ₗₘ **proceso id** { } **generar pacientes cada** *T unidad* **hacia id** ; *sentencias*	E → T
⇒ₗₘ **proceso id** { } **generar pacientes cada** *F unidad* **hacia id** ; *sentencias*	T → F
⇒ₗₘ **proceso id** { } **generar pacientes cada num** *unidad* **hacia id** ; *sentencias*	F → num
⇒ₗₘ **proceso id** { } **generar pacientes cada num minutos hacia id** ; *sentencias*	unidad → minutos
⇒ₗₘ **proceso id** { } **generar pacientes cada num minutos hacia id** ;	sentencias → ε

A.2 Declaración de recurso

Entrada: `recurso medicos = 3;`

decl_recurso

⇒ₗₘ **recurso id** = *E* ;	decl_recurso → recurso id = E ;
⇒ₗₘ **recurso id** = *T* ;	E → T
⇒ₗₘ **recurso id** = *F* ;	T → F
⇒ₗₘ **recurso id** = **num** ;	F → num

Entrada: `recurso camas = 2 * 10;`

decl_recurso

$\Rightarrow_{lm}$ **recurso id** = *E* ;	decl_recurso → recurso id = E ;
$\Rightarrow_{lm}$ **recurso id** = *T* ;	E → T
$\Rightarrow_{lm}$ **recurso id** = *T* * *F* ;	T → T * F
$\Rightarrow_{lm}$ **recurso id** = *F* * *F* ;	T → F
$\Rightarrow_{lm}$ **recurso id** = **num** * *F* ;	F → num
$\Rightarrow_{lm}$ **recurso id** = **num** * **num** ;	F → num

Entrada: `recurso sillas = medicos + 4;`

decl_recurso

$\Rightarrow_{lm}$ **recurso id** = *E* ;	decl_recurso → recurso id = E ;
$\Rightarrow_{lm}$ **recurso id** = *E* + *T* ;	E → E + T
$\Rightarrow_{lm}$ **recurso id** = *T* + *T* ;	E → T
$\Rightarrow_{lm}$ **recurso id** = *F* + *T* ;	T → F
$\Rightarrow_{lm}$ **recurso id** = **id** + *T* ;	F → id
$\Rightarrow_{lm}$ **recurso id** = **id** + *F* ;	T → F
$\Rightarrow_{lm}$ **recurso id** = **id** + **num** ;	F → num

Entrada: `recurso boxes = (2 + 3) * 2;`

decl_recurso

$\Rightarrow_{lm}$ **recurso id** = *E* ;	decl_recurso → recurso id = E ;
$\Rightarrow_{lm}$ **recurso id** = *T* ;	E → T
$\Rightarrow_{lm}$ **recurso id** = *T* * *F* ;	T → T * F
$\Rightarrow_{lm}$ **recurso id** = *F* * *F* ;	T → F
$\Rightarrow_{lm}$ **recurso id** = (*E*) * *F* ;	F → (E)
$\Rightarrow_{lm}$ **recurso id** = (*E* + *T*) * *F* ;	E → E + T
$\Rightarrow_{lm}$ **recurso id** = (*T* + *T*) * *F* ;	E → T
$\Rightarrow_{lm}$ **recurso id** = (*F* + *T*) * *F* ;	T → F
$\Rightarrow_{lm}$ **recurso id** = (**num** + *T*) * *F* ;	F → num
$\Rightarrow_{lm}$ **recurso id** = (**num** + *F*) * *F* ;	T → F
$\Rightarrow_{lm}$ **recurso id** = (**num** + **num**) * *F* ;	F → num
$\Rightarrow_{lm}$ **recurso id** = (**num** + **num**) * **num** ;	F → num

A.3 Declaración de proceso (con etapas)

Entrada: `proceso vacio { }`

decl_proceso

$\Rightarrow_{lm}$ **proceso id** { *etapas* }	decl_proceso → proceso id { etapas }
$\Rightarrow_{lm}$ **proceso id** { }	etapas → ε

Entrada: `proceso atencion { etapa consulta usa medicos durante 15 minutos; }`

decl_proceso

⇒lm **proceso id** { *etapas* }	decl_proceso → proceso id { etapas }
⇒lm **proceso id** { *decl_etapa etapas* }	etapas → decl_etapa etapas
⇒lm **proceso id** { **etapa id usa id durante** *E unidad* ; *etapas* }	decl_etapa → etapa id usa id durante E unidad ;
⇒lm **proceso id** { **etapa id usa id durante** *T unidad* ; *etapas* }	E → T
⇒lm **proceso id** { **etapa id usa id durante** *F unidad* ; *etapas* }	T → F
⇒lm **proceso id** { **etapa id usa id durante num** *unidad* ; *etapas* }	F → num
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; *etapas* }	unidad → minutos
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; }	etapas → ε

Entrada: `proceso emergencia { etapa triaje usa enfermeros durante 5 minutos; etapa reanimacion usa medicos durante 1 hora; }`

decl_proceso

⇒lm **proceso id** { *etapas* }	decl_proceso → proceso id { etapas }
⇒lm **proceso id** { *decl_etapa etapas* }	etapas → decl_etapa etapas
⇒lm **proceso id** { **etapa id usa id durante** *E unidad* ; *etapas* }	decl_etapa → etapa id usa id durante E unidad ;
⇒lm **proceso id** { **etapa id usa id durante** *T unidad* ; *etapas* }	E → T
⇒lm **proceso id** { **etapa id usa id durante** *F unidad* ; *etapas* }	T → F
⇒lm **proceso id** { **etapa id usa id durante num** *unidad* ; *etapas* }	F → num
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; *etapas* }	unidad → minutos
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; *decl_etapa etapas* }	etapas → decl_etapa etapas
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; **etapa id usa id durante** *E unidad* ; *etapas* }	decl_etapa → etapa id usa id durante E unidad ;
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; **etapa id usa id durante** *T unidad* ; *etapas* }	E → T
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; **etapa id usa id durante** *F unidad* ; *etapas* }	T → F
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; **etapa id usa id durante num** *unidad* ; *etapas* }	F → num
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; **etapa id usa id durante num hora** ; *etapas* }	unidad → hora
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; **etapa id usa id durante num hora** ; }	etapas → ε

Entrada: `proceso alta { etapa tramite usa admision durante 10 / 2 minutos; }`

decl_proceso

⇒lm **proceso id** { *etapas* }	decl_proceso → proceso id { etapas }
⇒lm **proceso id** { *decl_etapa etapas* }	etapas → decl_etapa etapas
⇒lm **proceso id** { **etapa id usa id durante** *E unidad* ; *etapas* }	decl_etapa → etapa id usa id durante E unidad ;
⇒lm **proceso id** { **etapa id usa id durante** *T unidad* ; *etapas* }	E → T
⇒lm **proceso id** { **etapa id usa id durante** *T* / *F unidad* ; *etapas* }	T → T / F
⇒lm **proceso id** { **etapa id usa id durante** *F* / *F unidad* ; *etapas* }	T → F
⇒lm **proceso id** { **etapa id usa id durante num** / *F unidad* ; *etapas* }	F → num
⇒lm **proceso id** { **etapa id usa id durante num** / **num** *unidad* ; *etapas* }	F → num
⇒lm **proceso id** { **etapa id usa id durante num** / **num minutos** ; *etapas* }	unidad → minutos
⇒lm **proceso id** { **etapa id usa id durante num** / **num minutos** ; }	etapas → ε

A.4 Generación de llegadas

Entrada: `generar pacientes cada 5 minutos hacia atencion;`

sent_llegada

⇒lm **generar pacientes cada** *E unidad* **hacia id** ;	sent_llegada → generar pacientes cada E unidad hacia id ;
⇒lm **generar pacientes cada** *T unidad* **hacia id** ;	E → T
⇒lm **generar pacientes cada** *F unidad* **hacia id** ;	T → F
⇒lm **generar pacientes cada num** *unidad* **hacia id** ;	F → num
⇒lm **generar pacientes cada num minutos hacia id** ;	unidad → minutos

Entrada: `generar pacientes cada 1 hora hacia emergencia;`

sent_llegada

⇒lm **generar pacientes cada** *E unidad* **hacia id** ;	sent_llegada → generar pacientes cada E unidad hacia id ;
⇒lm **generar pacientes cada** *T unidad* **hacia id** ;	E → T
⇒lm **generar pacientes cada** *F unidad* **hacia id** ;	T → F
⇒lm **generar pacientes cada num** *unidad* **hacia id** ;	F → num
⇒lm **generar pacientes cada num hora hacia id** ;	unidad → hora

Entrada: `generar pacientes cada 2 * 3 minutos hacia triaje;`

sent_llegada

⇒lm **generar pacientes cada** *E unidad* **hacia id** ;	sent_llegada → generar pacientes cada E unidad hacia id ;
⇒lm **generar pacientes cada** *T unidad* **hacia id** ;	E → T
⇒lm **generar pacientes cada** *T* * *F unidad* **hacia id** ;	T → T * F
⇒lm **generar pacientes cada** *F* * *F unidad* **hacia id** ;	T → F
⇒lm **generar pacientes cada num** * *F unidad* **hacia id** ;	F → num
⇒lm **generar pacientes cada num** * **num** *unidad* **hacia id** ;	F → num
⇒lm **generar pacientes cada num** * **num minutos hacia id** ;	unidad → minutos

Entrada: `generar pacientes cada 0.5 horas hacia consulta_externa;`

sent_llegada

⇒lm **generar pacientes cada** *E unidad* **hacia id** ;	sent_llegada → generar pacientes cada E unidad hacia id ;
⇒lm **generar pacientes cada** *T unidad* **hacia id** ;	E → T
⇒lm **generar pacientes cada** *F unidad* **hacia id** ;	T → F
⇒lm **generar pacientes cada num** *unidad* **hacia id** ;	F → num
⇒lm **generar pacientes cada num horas hacia id** ;	unidad → horas

A.5 Simulación

Entrada: `simular durante 8 horas;`

sent_simular

⇒lm **simular durante** *E unidad* ;	sent_simular → simular durante E unidad ;
⇒lm **simular durante** *T unidad* ;	E → T
⇒lm **simular durante** *F unidad* ;	T → F
⇒lm **simular durante num** *unidad* ;	F → num
⇒lm **simular durante num horas** ;	unidad → horas

Entrada: `simular durante 90 minutos;`

sent_simular

⇒lm **simular durante** *E unidad* ;	sent_simular → simular durante E unidad ;
⇒lm **simular durante** *T unidad* ;	E → T
⇒lm **simular durante** *F unidad* ;	T → F
⇒lm **simular durante num** *unidad* ;	F → num
⇒lm **simular durante num minutos** ;	unidad → minutos

Entrada: `simular durante 1 hora;`

sent_simular

⇒lm **simular durante** *E unidad* ;	sent_simular → simular durante E unidad ;
⇒lm **simular durante** *T unidad* ;	E → T
⇒lm **simular durante** *F unidad* ;	T → F
⇒lm **simular durante num** *unidad* ;	F → num
⇒lm **simular durante num hora** ;	unidad → hora

Entrada: `simular durante 24 * 7 horas;`

sent_simular

⇒lm **simular durante** *E unidad* ;	sent_simular → simular durante E unidad ;
⇒lm **simular durante** *T unidad* ;	E → T
⇒lm **simular durante** *T* * *F unidad* ;	T → T * F
⇒lm **simular durante** *F* * *F unidad* ;	T → F
⇒lm **simular durante num** * *F unidad* ;	F → num
⇒lm **simular durante num** * **num** *unidad* ;	F → num
⇒lm **simular durante num** * **num horas** ;	unidad → horas

A.6 Consulta de métricas

Entrada: `mostrar pacientes_atendidos;`

sent_mostrar

⇒lm **mostrar id** ; — sent_mostrar → mostrar id ;

Entrada: `mostrar espera_promedio;`

sent_mostrar

⇒lm **mostrar id** ; — sent_mostrar → mostrar id ;

Entrada: `mostrar ocupacion(medicos);`

sent_mostrar

⇒lm **mostrar id** (**id**) ; — sent_mostrar → mostrar id (id) ;

Entrada: `mostrar ocupacion(camas);`

sent_mostrar

⇒lm **mostrar id** (**id**) ; — sent_mostrar → mostrar id (id) ;

A.7 Expresiones aritméticas

Entrada: `2 + 3 * 4`

E

⇒lm *E* + *T* — E → E + T

⇒lm *T* + *T* — E → T

⇒lm *F* + *T* — T → F

⇒lm **num** + *T* — F → num

⇒lm **num** + *T* * *F* — T → T * F

⇒lm **num** + *F* * *F* — T → F

⇒lm **num** + **num** * *F* — F → num

⇒lm **num** + **num** * **num** — F → num

Entrada: `(2 + 3) * 4`

E

⇒lm *T* — E → T

⇒lm *T* * *F* — T → T * F

⇒lm *F* * *F* — T → F

⇒lm (*E*) * *F* — F → (E)

⇒lm (*E* + *T*) * *F* — E → E + T

⇒lm (*T* + *T*) * *F* — E → T

⇒lm (*F* + *T*) * *F* — T → F

⇒lm (**num** + *T*) * *F* — F → num

⇒lm (**num** + *F*) * *F* — T → F

⇒lm (**num** + **num**) * *F* — F → num

⇒lm (**num** + **num**) * **num** — F → num

Entrada: `20 - 5 - 3`

E	
$\Rightarrow_{lm}$ *E* − *T*	E → E − T
$\Rightarrow_{lm}$ *E* − *T* − *T*	E → E − T
$\Rightarrow_{lm}$ *T* − *T* − *T*	E → T
$\Rightarrow_{lm}$ *F* − *T* − *T*	T → F
$\Rightarrow_{lm}$ **num** − *T* − *T*	F → num
$\Rightarrow_{lm}$ **num** − *F* − *T*	T → F
$\Rightarrow_{lm}$ **num** − **num** − *T*	F → num
$\Rightarrow_{lm}$ **num** − **num** − *F*	T → F
$\Rightarrow_{lm}$ **num** − **num** − **num**	F → num

Entrada: `medicos / 2`

E	
$\Rightarrow_{lm}$ *T*	E → T
$\Rightarrow_{lm}$ *T* / *F*	T → T / F
$\Rightarrow_{lm}$ *F* / *F*	T → F
$\Rightarrow_{lm}$ **id** / *F*	F → id
$\Rightarrow_{lm}$ **id** / **num**	F → num